P1155R3
More implicit moves

Published Proposal, 2019-06-17

Authors:

Arthur O'Dwyer

David Stone

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1155-more-implicit-moves.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1155-more-implicit-moves.html

---

Abstract

Programmers expect `return x;` to trigger copy elision; or, at worst, to *implicitly move* from `x` instead of copying. Occasionally, C++ violates their expectations and performs an expensive copy anyway. Based on our experience using Clang to diagnose unexpected copies in Chromium, Mozilla, and LibreOffice, we propose to change the standard so that these copies will be replaced with *implicit moves*.

Table of Contents

§ 1. Changelog

- R3: Eliminate discussion and move to CWG wording review.
- R2: Added discussion of `return ++x`.
- R1: Added discussion of `return x += y`.

§ 2. Background

Each version of C++ has improved the efficiency of returning objects by value. By the middle of the last decade, copy elision was reliable (if not technically guaranteed) in situations like this:

```
Widget one() {
    return Widget();  // copy elision
}
Widget two() {
    Widget result;
    return result;  // copy elision
}
```

In C++11, a completely new feature was added: a change to overload resolution which I will call *implicit move*. Even when copy elision is impossible, the compiler is sometimes required to *implicitly move* the `return` statement's operand into the result object:

```
std::shared_ptr<Base> three() {
    std::shared_ptr<Base> result;
    return result;  // copy elision
}
std::shared_ptr<Base> four() {
    std::shared_ptr<Derived> result;
    return result;  // no copy elision, but implicitly moved (not copied)
}
```

The wording for this optimization was amended by [CWG1579]. N4762's wording in [class.copy.elision]/3 says:

> In the following copy-initialization contexts, a move operation might be used instead of a copy operation:
>
> - If the *expression* in a `return` statement is a (possibly parenthesized) *id-expression* that names an object with automatic storage duration declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
> - if the operand of a *throw-expression* is the name of a non-volatile automatic object (other than a **function** or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one),
>
> overload resolution to select the constructor for the copy is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, or if the type of the first parameter of the selected **constructor** is not an **rvalue reference** to **the object's** type (possibly cv-qualified), overload resolution is performed again, considering the object as an lvalue.

> Note: Post-Kona draft N4810 extends this wording to cover `co_return` statements too. I don't understand coroutines very well, but I don't know of any reason why P1155's changes should not apply to `co_return` just as well as to `return` and `throw`. The proposed wording below reflects my beliefs.

The highlighted phrases above indicate places where the wording diverges from a naïve programmer's intuition. Consider the following examples...

§ 2.1. Throwing is pessimized

Throwing is pessimized because of the highlighted word **function** [parameter].

```
void five() {
    Widget w;
    throw w;  // non-guaranteed copy elision, but implicitly moved (never c
}
Widget six(Widget w) {
    return w;  // no copy elision, but implicitly moved (never copied)
}
void seven(Widget w) {
    throw w;  // no copy elision, and no implicit move (the object is copie
}
```

> Note: The comment in `seven` matches the current Standard wording, and matches the behavior of GCC. Most compilers (Clang 4.0.1+, MSVC 2015+, ICC 16.0.3+) already do this implicit move.

§ 2.2. Non-constructor conversion is pessimized

Non-constructor conversion is pessimized because of the highlighted word **constructor** .

```
struct From {
    From(Widget const &);
    From(Widget&&);
};

struct To {
    operator Widget() const &;
    operator Widget() &&;
};

From eight() {
    Widget w;
    return w;  // no copy elision, but implicitly moved (never copied)
}
Widget nine() {
    To t;
    return t;  // no copy elision, and no implicit move (the object is copi
}
```

§ 2.3. By-value sinks are pessimized

By-value sinks are pessimized because of the highlighted phrase **rvalue reference** .

```
struct Fish {
    Fish(Widget const &);
    Fish(Widget&&);
};

struct Fowl {
    Fowl(Widget);
};

Fish ten() {
    Widget w;
    return w;  // no copy elision, but implicitly moved (never copied)
}
Fowl eleven() {
    Widget w;
    return w;  // no copy elision, and no implicit move (the Widget object
}
```

Note: The comment in `eleven` matches the current Standard wording, and matches the behavior of Clang, ICC, and MSVC. One compiler (GCC 5.1+) already does this implicit move.

§ 2.4. Slicing is pessimized

Slicing is pessimized because of the highlighted phrase **the object's** .

```
std::shared_ptr<Base> twelve() {
    std::shared_ptr<Derived> result;
    return result;  // no copy elision, but implicitly moved (never copied)
}
Base thirteen() {
    Derived result;
    return result;  // no copy elision, and no implicit move (the object is
}
```

Note: The comment in `thirteen` matches the current Standard wording, and matches the behavior of Clang and MSVC. Some compilers (GCC 8.1+, ICC 18.0.0+) already do this implicit move.

We propose to remove all four of these unnecessary limitations.

§ 3. Proposed wording relative to N4810

Modify [class.copy.elision]/3 as follows:

> In the following copy-initialization contexts, a move operation might be used instead of a copy operation:
>
> - If the *expression* in a `return` or `co_return` statement is a (possibly parenthesized) *id-expression* that names an object with automatic storage duration declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
> - if the operand of a *throw-expression* is the name of a non-volatile automatic object (other than a ~~function or~~ catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one),
>
> overload resolution to select the constructor for the copy or the `return_value` overload to call is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, ~~or if the type of the first parameter of the selected constructor or the `return_value` overload is not an rvalue reference to the object's type (possibly cv-qualified),~~ overload resolution is performed again, considering the object as an lvalue. [*Note:* This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor or the `return_value` overload to be called if elision is not performed, and the selected constructor or ~~the~~ `return_value` overload must be accessible even if the call is elided. —*end note*]

§ 4. Acknowledgments

- Thanks to Lukas Bergdoll for his copious feedback.
- Thanks to David Stone for [P0527], and for offering to shepherd P1155R0 at the San Diego WG21 meeting (November 2018).
- Thanks to Barry Revzin (see [Revzin]) for pointing out the "By-value sinks" case.

§ References

§ Informative References

[CWG1579]

Jeffrey Yasskin. Return by converting move constructor. October 2012. URL: http://open-std.org/JTC1/SC22/WG21/docs/cwg_defects.html#1579

[P0527]

David Stone. Implicitly move from rvalue references in return statements. November 2017. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0527r1.html

[Revzin]

Barry Revzin; Howard Hinnant; Arthur O'Dwyer. std-proposals thread: By-value sinks. August 2018. URL: https://groups.google.com/a/isocpp.org/d/msg/std-proposals/eeLS8vI05nM/_BP-8YTPDAAJ